

CET322: Introduction to Computer Virtualization

Lecture 2



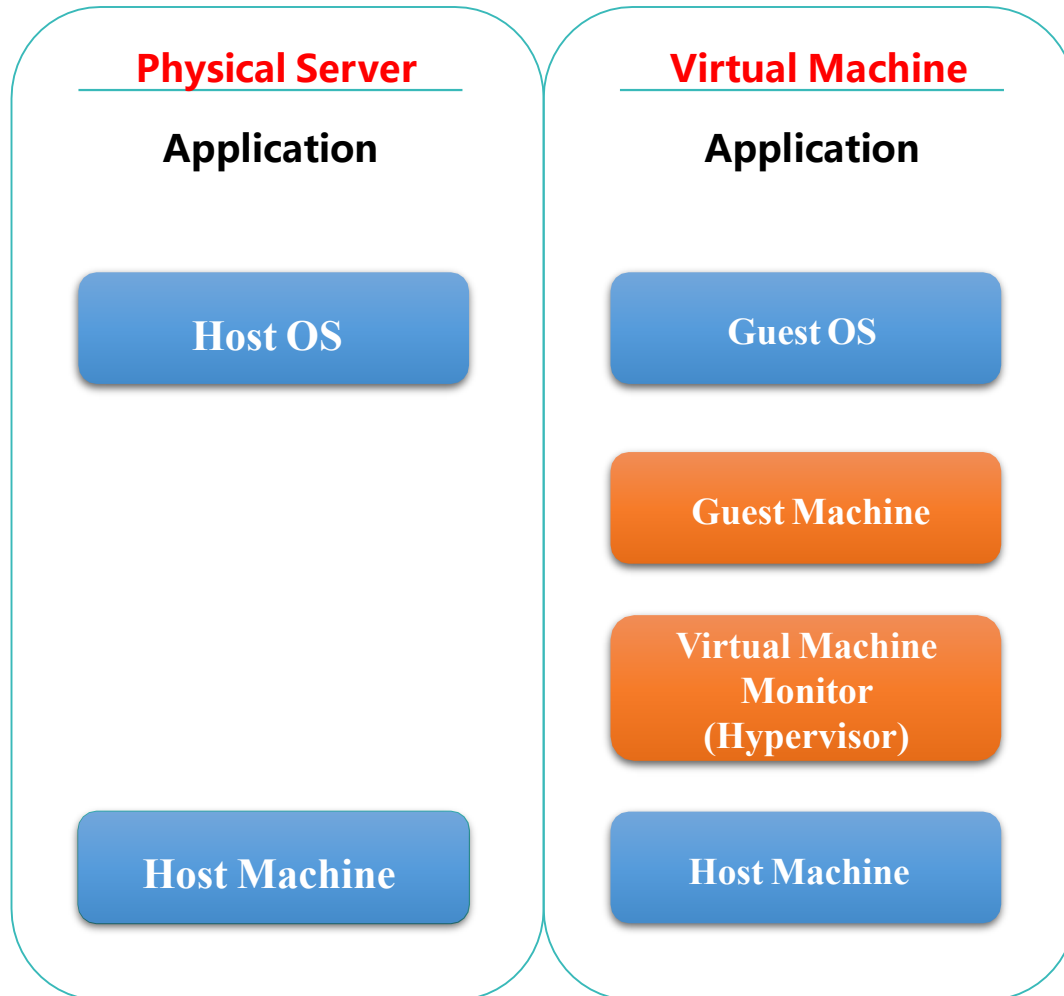
Outline

- Introduction to Virtual Machine
- Virtual Machine Manager
- Characteristics of Computer Virtualization
- Operation Mode
- Example VMs

Introduction to Virtualization

- Fundamental idea – **abstract** hardware of a single computer into several different execution environments
 - Similar to layered approach
 - Virtualization layer creates virtual system (**virtual machine**, or **VM**) on which operation systems or applications can run
- Several components
 1. **Host** – underlying hardware system
 2. **Virtual machine manager (VMM)** or **hypervisor** – creates and runs virtual machines by providing interface that is **identical** to the host
 - (Except in the case of paravirtualization)
 3. **Guest** – process provided with virtual copy of the host
 - Usually an operating system
- Through the VMM Single physical machine can run multiple operating systems concurrently, each in its own virtual machine

Virtual Machine (Cont.)



Guest OS:

- Operating system running in a virtual machine (VM)

Guest Machine:

- Virtual machine created through virtualization

Hypervisor:

- Virtualization software layer, or
Virtual Machine Monitor (VMM)

Host OS:

- Operating system running in a physical machine

Host Machine:

- Physical machine

Virtual Machine (Cont.)

- VMs provide a way to run **multiple OS** simultaneously on a **single physical machine**.
- Each VM operates as an **independent entity**, with its own **virtualized hardware resources** (*virtual CPU, memory, network interface, and storage*).
- The **physical machine** on which the VM runs is called the **host machine**, while the **VM** itself is referred to as the **guest machine**.
- The **hypervisor** treats the **physical machine's resources** as a **pool** that can be **allocated** and **shared among multiple VMs**.
- VMs can run **various operating systems, applications, and software**, just like **physical computers**.
- VMs can be **easily moved** between **different hypervisors or physical machines**, providing benefits such as lowered downtime, scalability, and security advantages .
- Virtual machines are commonly used for various purposes, including **backup solutions, workload distribution, security forensics, and software development and testing**.

VM Process

Hosted hypervisors:

Examples: VMware Workstation, VirtualBox, Parallels.

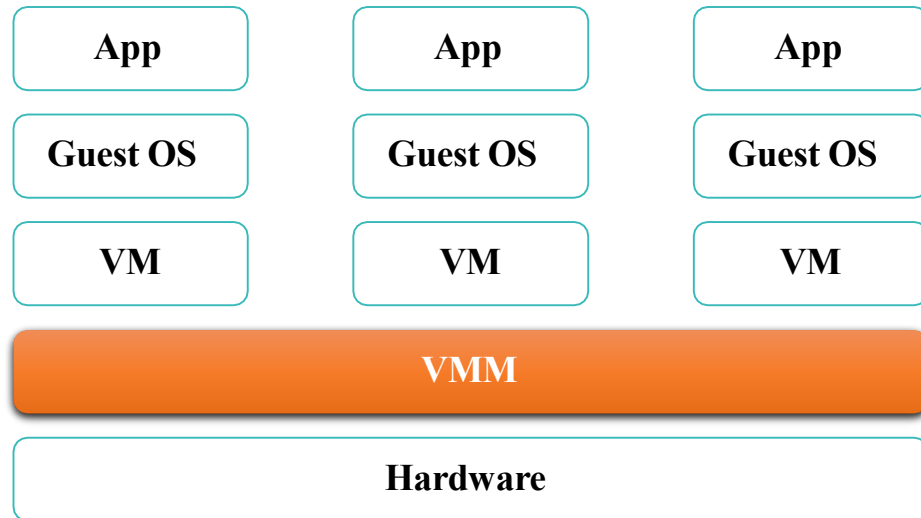
- The hypervisor runs on top of a host OS (Windows/macOS/Linux) as one or more user-space processes.
- Each VM is represented by one main process plus several threads (e.g., one thread per vCPU, I/O threads, emulator threads).
- The host OS gives those processes/threads normal OS treatment:
 - They have a **PCB** (Process Control Block) / task_struct (Linux) like any other process/thread.
 - They are placed on the **host scheduler's** run queues and **preempted** / time-sliced like everything else.
- Inside that scheduled time slice, the guest OS does its own scheduling of guest processes onto vCPUs.
→ This yields two-level scheduling: host schedules vCPU threads; guest schedules guest processes on those vCPUs.

What exactly is “abstracting hardware into several different execution environments”?

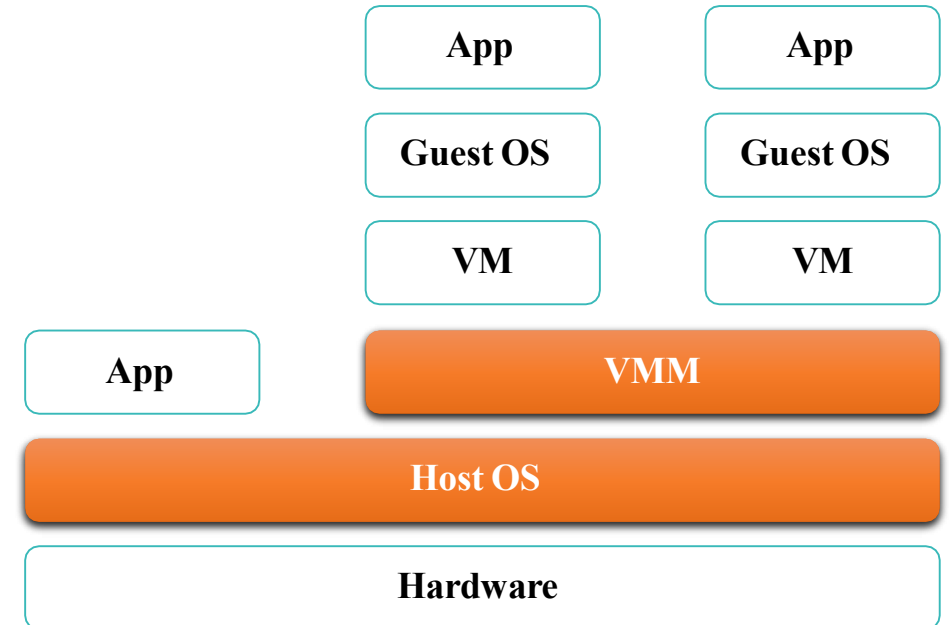
- CPU: The VM “thinks” it has its own CPUs (vCPUs). The hypervisor schedules real CPU time for each vCPU (like time-sharing).
- Memory: Each VM sees a private memory space. Under the hood, the hypervisor uses techniques like second-level address translation (e.g., EPT/NPT) to map guest physical memory to real machine memory safely and efficiently.
- Storage: A VM’s “disk” is often just a large file (or a logical volume) on the host; to the guest OS it looks like a real disk.
- Devices & I/O: Network interfaces, GPUs, and other devices can be emulated, paravirtualized (high-performance virtual drivers), or sometimes passed through directly (e.g., SR-IOV, VT-d/IOMMU) for near-native speed.

Hypervisors: Most Common Types and Functionality

- **Hypervisors**, also known as **virtual machine monitors/managers (VMMs)**, are software that **create** and **run** **virtual machines (VMs)**.
- They enable computer virtualization by separating the physical hardware from the guest operating systems on it.
- There are two main types of hypervisors: - Type 1 and Type 2.



**Bare-Metal Virtualization
(Type 1)**



**Hosted Virtualization
(Type 2)**

Implementation of VMMs

Vary greatly, with options including:

- **Type 0 hypervisors** - Hardware-based solutions that provide support for virtual machine creation and management via firmware
 - IBM LPARs and Oracle LDOMs are examples
- **Type 1 hypervisors** - Operating-system-like software built to provide virtualization
 - Including VMware ESX, Joyent SmartOS, and Citrix XenServer
- **Type 1 hypervisors** – Also includes general-purpose operating systems that provide standard functions as well as VMM functions
 - Including Microsoft Windows Server with HyperV and RedHat Linux with KVM
- **Type 2 hypervisors** - Applications that run on standard operating systems but provide VMM features to guest operating systems
 - Including VMware Workstation and Fusion, Parallels Desktop, and Oracle VirtualBox

Type 0 Hypervisor

- Definition: The earliest generation of hypervisors, tightly integrated into the mainframe's operating system or firmware.
- How it works: Runs directly on hardware but is essentially part of the OS kernel itself. Guest OSes run as if they were applications managed by the supervisor.
- Examples: IBM's PR/SM (Processor Resource/System Manager), IBM VM/370.
- Advantages:
 - Very stable and mature — mainframes ran on these for decades.
 - Deep hardware integration → high reliability and performance.
- Disadvantages:
 - Proprietary, limited to specific hardware (e.g., IBM mainframes).
 - Not portable for general computing environments.

Type 1 Hypervisors (Bare-metal Hypervisors)

- Definition: Installed directly on the physical hardware (no underlying host OS).
- How it works: They interact directly with the system's hardware and manage guest VMs.
- Examples: VMware ESXi, Microsoft Hyper-V, Xen, KVM.
- Advantages:
 - High performance and efficiency (no “middleman” OS).
 - More secure — fewer layers means a smaller attack surface.
 - Common in data centers, enterprise cloud platforms, and production servers.
- Disadvantages:
 - More complex to set up and manage.
 - Limited hardware compatibility compared to running on a full OS.

Type 2 Hypervisors (Hosted Hypervisors)

- Definition: Installed on top of a host operating system (Windows, Linux, macOS).
- How it works: The host OS handles hardware interactions, while the hypervisor manages VMs.
- Examples: Oracle VirtualBox, VMware Workstation, Parallels Desktop.
- Advantages:
 - Easy to install and use (just like installing an app).
 - Good for testing, development, education, and personal use.
- Disadvantages:
 - Slower than Type 1 (extra overhead from the host OS).
 - Less secure — vulnerabilities in the host OS can affect the VMs.
 - Not ideal for large-scale production environments.

Special/Hybrid Categories

Some modern hypervisors blur the line between Type 1 and Type 2:

- KVM (Kernel-based Virtual Machine): Runs on Linux but is considered Type 1, because once loaded, it gives direct hardware access like a bare-metal hypervisor.
- Microsoft Hyper-V: Often installed on Windows, but under the hood it works as a Type 1 hypervisor (Windows itself runs as a VM on top of it).
- Containers vs. Hypervisors: Containers (Docker, Kubernetes) aren't hypervisors — they share the host OS kernel rather than virtualizing hardware — but are often compared because they solve similar problems in a lighter way.

Comparison Table

Feature	Type 0 (Firmware/Mainfram)	Type 1 (Bare-metal)	Type 2 (Hosted)	Hybrid
Installed on	Mainframe OS/firmware	Hardware directly	Host OS	Host OS + Kernel
Performance	Very High (mainframe-optimized)	High	Moderate	High
Use Case	Mainframes, IBM systems	Cloud, Data centers	Testing, Development	Mixed workloads
Examples	IBM PR/SM, VM/370	ESXi, Xen, Hyper-V Core	VirtualBox, VMware Workstation	KVM, Hyper-V (Desktop)
Complexity	Proprietary, specialized	Medium-High	Low	Medium

Characteristics of Computer Virtualization

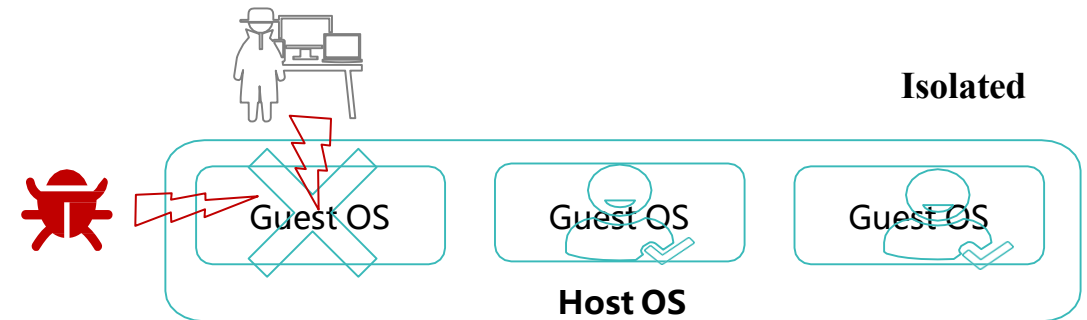
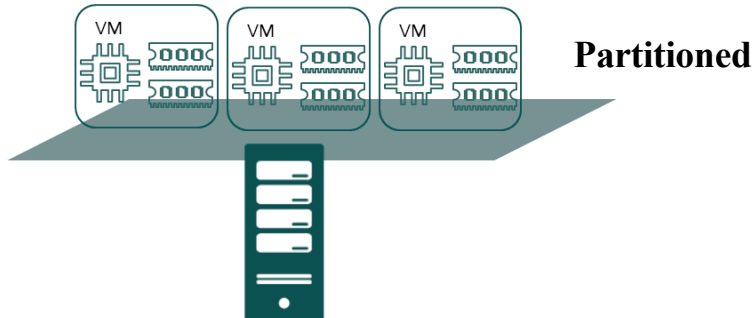
❑ Computer virtualization exhibits several key characteristics, including:

1. Partitioned

- ❑ Virtualization allows for the partitioning of physical resources, such as CPU, memory, and storage, into multiple virtual machines.
- ❑ Each virtual machine operates independently and can be allocated a specific portion of the available resources.

2. Resource Isolation

- ❑ Virtualization provides isolation between different virtual machines.
- ❑ Each virtual machine operates in its own dedicated environment, separate from other virtual machines.
- ❑ This isolation ensures that the activities of one virtual machine do not impact others.



Characteristics of Computer Virtualization (Cont.)

3. Encapsulated

- ❑ Each VM is saved as a group of hardware-independent files, including the hardware configuration, BIOS configuration, memory status, disk status, and CPU status.
- ❑ Encapsulation is the most important feature for VM migration and virtualization. This is because encapsulating a VM as a set of hardware-independent files makes VM migration and hot swap possible.

4. Hardware independence

- ❑ After a VM is encapsulated into a group of files, the VM is completely independent from its underlying hardware.
- ❑ You can migrate the VM by copying the VM device file, configuration file, or disk file to another host.



Characteristics of Computer Virtualization (Cont.)

5. Load Balancing

- ❑ Virtualization platforms often include load balancing capabilities.
- ❑ These capabilities distribute incoming requests or workloads across multiple virtual machines or servers, ensuring optimal utilization and preventing overload.

6. Performance Tuning

- ❑ Virtualization allows for fine-tuning of virtual machine performance by adjusting resource allocation and configuration settings.
- ❑ This capability enables the implementation of quality-of-service (QoS) infrastructure, ensuring that critical workloads receive the necessary resources .

- ❑ These characteristics contribute to the flexibility, efficiency, and scalability offered by computer virtualization technologies in various computing environments.
- ❑ They enable organizations to optimize resource utilization, improve workload management, and enhance overall system performance.

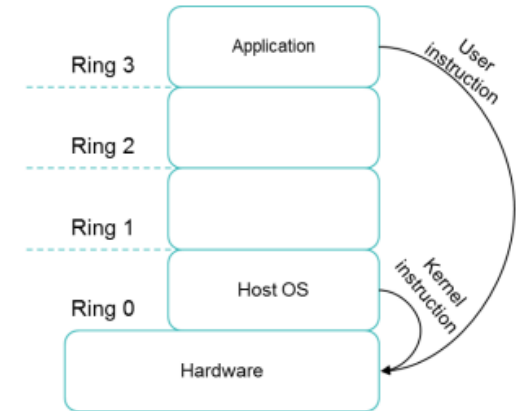
Compute Virtualization Modes of Operation

- ❑ Compute virtualization includes CPU virtualization, memory virtualization, and I/O virtualization.

CPU Virtualization

- ❑ The hierarchical protection domains of CPUs, often called protection rings:

- There are four rings: Ring 0, Ring 1, Ring 2, and Ring 3, which is a hierarchy of control from the most to least privilege.
- Ring 0 has direct access to the hardware.
- Generally, only the OS and driver have this privilege.
- Ring3 has the least privilege.
- All programs have the privilege of Ring 3.
- To protect the computer, some dangerous instructions can only be executed by the OS, preventing malicious software from randomly calling hardware resources. For example, if a program needs to enable a camera, the program must request a Ring 0 driver to do that on its behalf. Otherwise, the operation will be rejected.



- ❑ The OS on a common host sends two types of instructions:

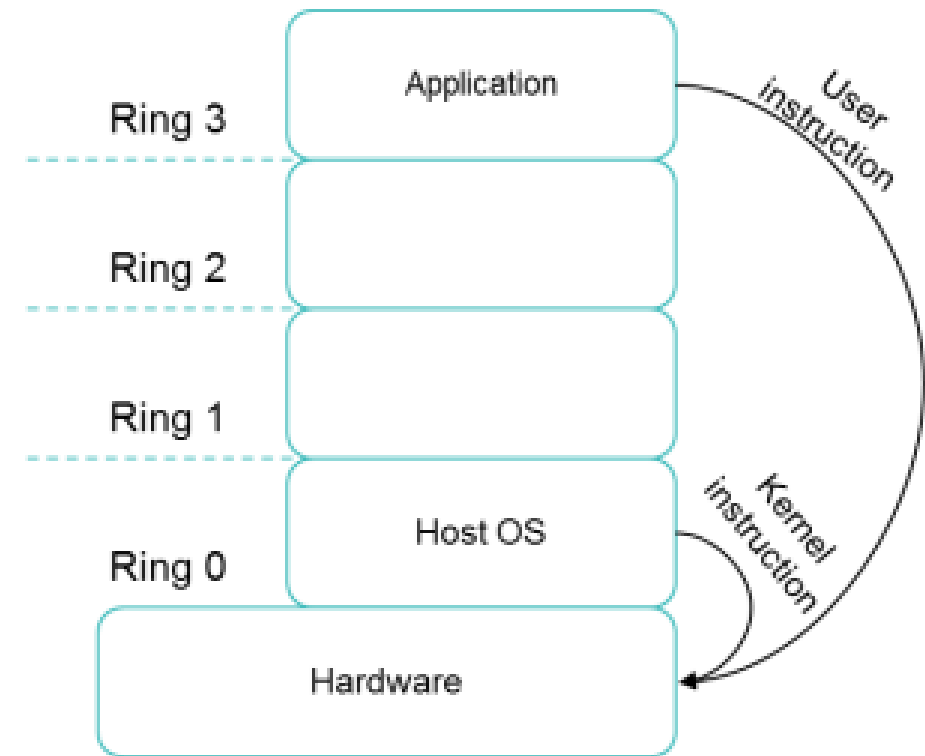
1. Privileged Instructions: instructions used to manipulate and manage key system resources. These instructions can be executed by programs of the highest privilege level, that is, Ring 0.
2. Common Instructions: instructions can be executed by programs of the common privilege level, that is, Ring 3.

Modes of Operation in a Virtual Machine System (Cont.)

On modern CPUs, we usually distinguish between 2 types of privilege:

- Host CPU Modes (Classic Hardware)

- Kernel Mode (Ring 0)
 - Full access to hardware.
 - Normally reserved for the OS kernel.
- User Mode (Ring 3)
 - Limited access.
 - Used for applications.
 - Privileged instructions cause a trap to kernel mode.



CPU Protection Rings (Ring 0 /Kernel Mode → Ring 3 / User Mode)

• Ring 0 – Kernel Mode (Highest Privilege)

- Full access to hardware resources (CPU instructions, memory, I/O devices).
- Can execute privileged instructions (e.g., change page tables, disable interrupts, perform I/O).
- The Operating System kernel (Linux kernel, Windows NT kernel, etc.) runs here.
- If Ring 0 code crashes, the whole system can fail (blue screen, kernel panic).

Ring 1 – Device Drivers / OS Services

- Historically intended for low-level services like device drivers.
- In practice, modern OSs often run drivers in Ring 0 for performance reasons, (Rarely used).

Ring 2 – More Restricted OS Services

- Meant for less trusted kernel services (e.g., networking stacks).
- Not widely used in modern systems.

Ring 3 – User Mode (Lowest Privilege)

Restricted access — cannot directly interact with hardware.

If a program in Ring 3 wants to use hardware (e.g., memory), it request service from Ring 0 via a system call.

Examples: Applications like Word if it crashes, only the app fails — the OS keeps running.

Comparison Table

Ring	Privilege Level	Who Runs Here	Access
0	Highest (Kernel)	OS Kernel	Full hardware access
1	High	Device drivers (theoretical)	Limited
2	Medium	OS services (theoretical)	Limited
3	Lowest (User mode)	Applications	No direct hardware access

Building Block – Trap and Emulate

- Dual mode CPU means guest executes in user mode
 - Kernel runs in kernel mode
 - Not safe to let guest kernel run in kernel mode too
 - So *VM* needs to run in two modes – *virtual user mode and virtual kernel mode*
 - Both of which run in real user mode
- Actions in guest that usually cause switch to kernel mode must cause switch to virtual kernel mode

Compute Virtualization (sensitive instructions)

- In a virtualized environment, there is another special type of instruction called *sensitive instruction (or trap instruction)*.
- A sensitive instruction is used for changing the operating mode of a VM or the state of a host machine.
- The instruction is handled by VMM after a privilege instruction that originally needs to be run in Ring 0 in the guest OS is deprived of the privilege.

How the Trap Signal Works in Virtual Machines

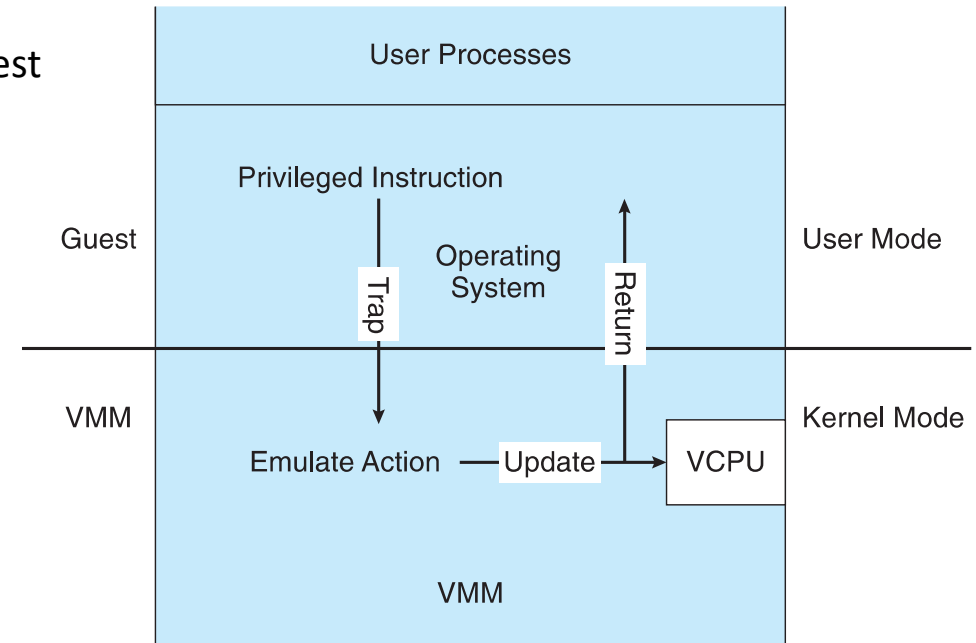
A trap is a CPU signal (like an interrupt) that occurs when a privileged instruction is executed in a non-privileged mode.

- In a real physical system:
 - Certain instructions (e.g., modifying page tables, accessing I/O devices) are privileged.
 - If a user program tries to run them, the CPU generates a trap and hands control to the operating system.
- In a virtualized system:
 - A guest OS thinks it runs in kernel mode, but in reality, it's running in a restricted mode (so it can't directly touch hardware).
- When the guest OS executes a privileged instruction (like I/O or memory management), the CPU traps to the VMM (Virtual Machine Monitor / Hypervisor).
- The hypervisor then decides:
 - Emulate the instruction (software simulation).
 - Pass through the request to the actual hardware.
 - Deny the operation (if unsafe or isolated).

This mechanism is often described as Trap-and-Emulate — it's fundamental to Type 1 and Type 2 hypervisors.

Trap-and-Emulate (Cont.)

- How does switch from virtual user mode to virtual kernel mode occur?
 - Attempting a privileged instruction in user mode causes an error -> trap
 - VMM gains control, analyzes error, executes operation as attempted by guest
 - Returns control to guest in user mode
 - Known as **trap-and-emulate**
 - Most virtualization products use this at least in part
- User mode code in guest runs at same speed as if not a guest
- But kernel mode privilege mode code runs slower due to trap-and-emulate
 - Especially a problem when multiple guests running, each needing trap-and-emulate
- CPUs adding hardware support, mode CPU modes to improve virtualization performance



System Call (Syscall)

SYSCALL Definition: A controlled way for a user program (Ring 3) to request a service from the OS kernel (Ring 0).

Triggered by: The program itself (software request).

Mechanism:

- 1.App executes a **special instruction** (like `syscall` on x86-64 or `int 0x80` on older x86).
- 2.CPU switches from **user mode** → **kernel mode**.
- 3.Kernel executes the requested service (e.g., read a file, allocate memory).
- 4.Control returns to the user program.

Examples:

- `read()`, `write()`, `fork()`, `exec()`, `open()`, `close()` in Unix/Linux.
- Windows API functions like `CreateFile()`, `ReadFile()`.

Analogy: You're at a bank → you fill out a form (syscall) to ask the teller (kernel) for service.

Interrupt

Interrupt Definition: An asynchronous signal to the CPU that something needs immediate attention.

Triggered by:

- Hardware interrupts: From devices (keyboard press, mouse movement, disk finished I/O).
- Software interrupts: Raised by software (like exceptions, divide-by-zero, invalid memory).

Mechanism:

- Device or event raises an interrupt signal.
- CPU pauses current execution, jumps to an Interrupt Handler in the kernel.
- Kernel handles the event, then resumes the original program.

Examples:

- Hardware: Pressing a key (keyboard interrupt 0x09), receiving a network packet.
- Software: Divide-by-zero → CPU raises interrupt to kernel.

Analogy: You're at the bank → suddenly the fire alarm rings (interrupt) → everyone must respond immediately, even before finishing the current work.

Key Differences: Syscall vs Interrupt

Aspect	System Call	Interrupt
Who triggers it?	User program (software request).	Hardware device or CPU itself (event).
Synchronous/Async	Synchronous (happens exactly where program requests it).	Asynchronous (can happen anytime).
Purpose	To ask kernel for a service (I/O, memory, process mgmt).	To notify CPU of an urgent event (hardware/software).
Examples	<code>open()</code> , <code>read()</code> , <code>write()</code> .	Keyboard press, divide-by-zero, timer interrupt.
Instruction used	<code>syscall</code> , <code>sysenter</code> , <code>int 0x80</code> .	Hardware signal or CPU exception.

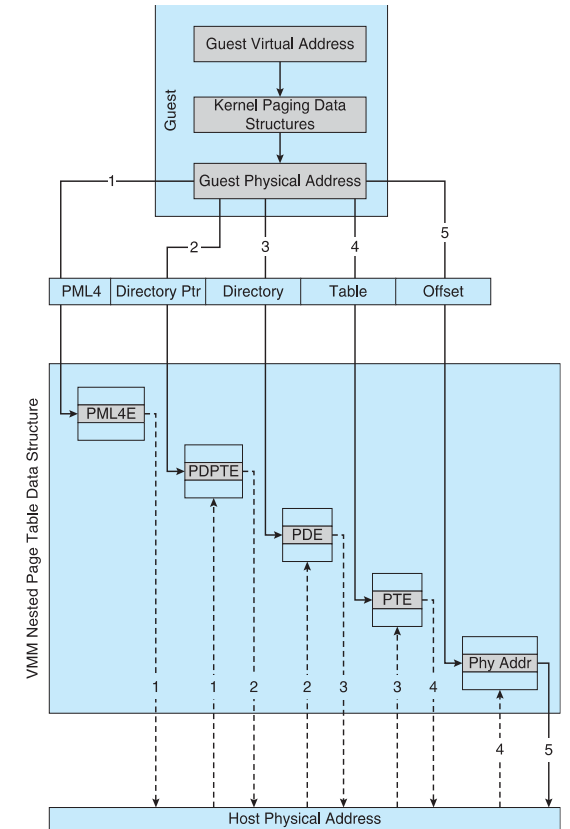
VM Scheduling

Across Multiple OSES (via Virtualization / Containers)

- On a server running virtual machines (with a hypervisor), each VM has its own OS scheduler.
- At the same time, the hypervisor itself has a scheduler to decide which VM gets CPU time.
- Example:
 - A Linux VM with CFS scheduler.
 - A Windows VM with its round-robin scheduler.
 - The hypervisor (KVM, VMware ESXi, Hyper-V) schedules both VMs on the host CPUs.
- Result: multiple schedulers running in parallel at different layers.

Memory management

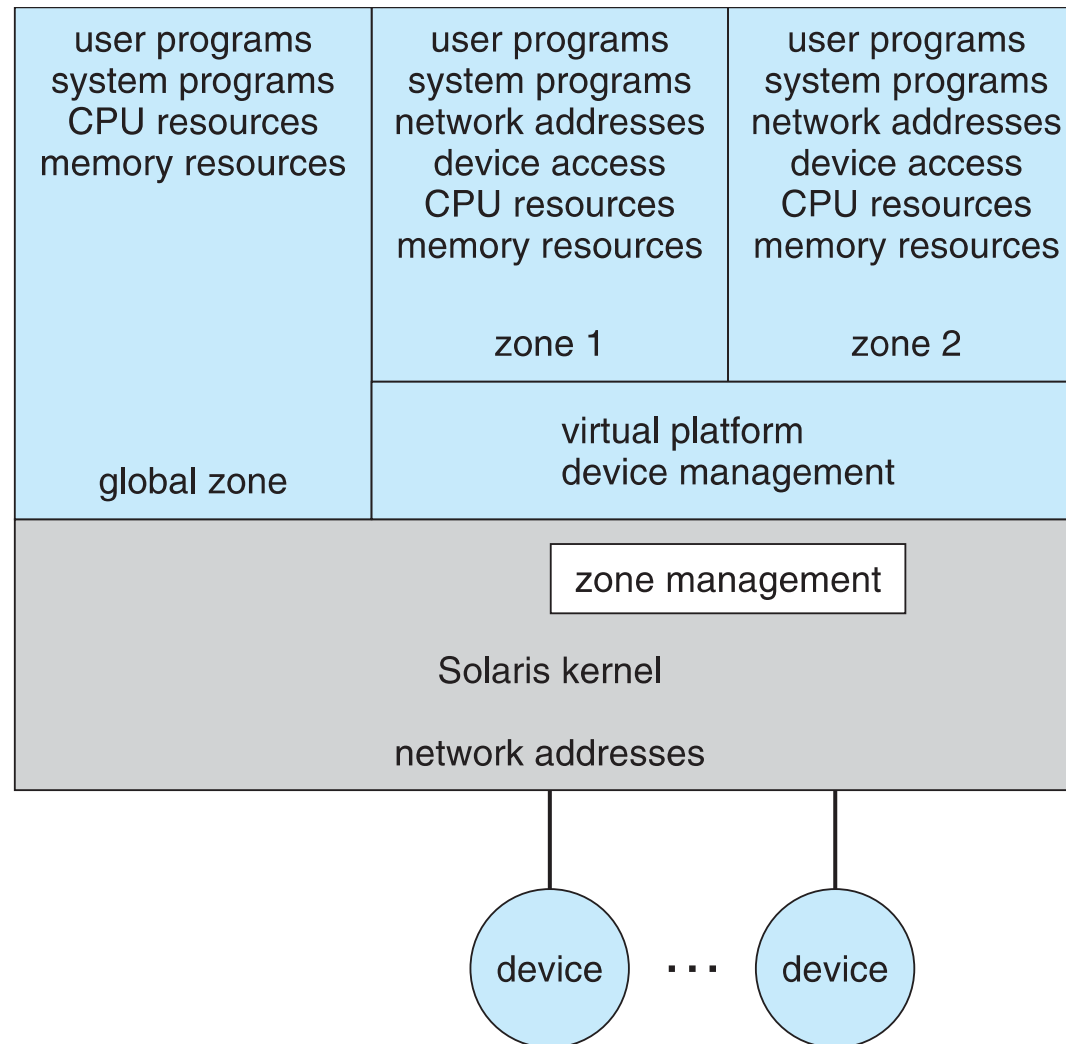
- Memory management another general challenge to VMM implementations
- How can VMM keep page-table state for both guests believing they control the page tables and VMM that does control the tables?
- Common method (for trap-and-emulate and binary translation) is **nested page tables (NPTs)**
 - Each guest maintains page tables to translate virtual to physical addresses
 - VMM maintains per guest NPTs to represent guest's page-table state
 - Just as VCPU stores guest CPU state
 - When guest on CPU -> VMM makes that guest's NPTs the active system page tables
 - Guest tries to change page table -> VMM makes equivalent change to NPTs and its own page tables
 - Can cause many more TLB misses -> much slower performance



Types of VMs – Application Containment

- Some goals of virtualization are segregation of apps, performance and resource management, easy start, stop, move, and management of them
- Can do those things without full-fledged virtualization
 - If applications compiled for the host operating system, don't need full virtualization to meet these goals
- Oracle **containers** / **zones** for example create virtual layer between OS and apps
 - Only one kernel running – host OS
 - OS and devices are virtualized, providing resources within zone with impression that they are only processes on system
 - Each zone has its own applications; networking stack, addresses, and ports; user accounts, etc
 - CPU and memory resources divided between zones
 - Zone can have its own scheduler to use those resources

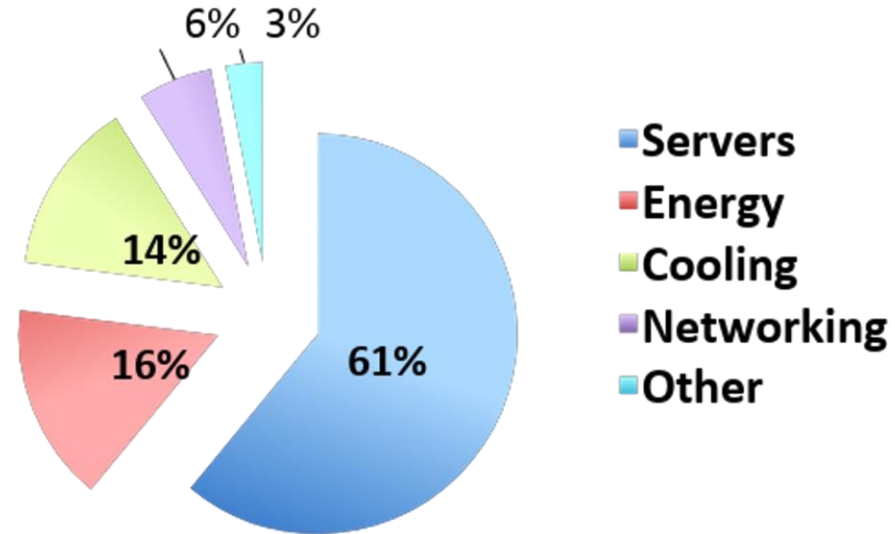
Containers / Zones



Total Cost of Ownership (TCO)

- **TCO = capital (CapEx) + operational (OpEx) expenses**
- Operators perspective
 - CapEx: building, generators, A/C, compute/storage/net HW Including spares, amortized over 3 – 15 years
 - OpEx: electricity (5-7c/KWh), repairs, people, WAN, insurance, ...
- Users perspective
 - CapEx: cost of long term leases on HW and services OpEx: pay per use cost on HW and services, people

Operator's TCO Example



[Source: James Hamilton]

Hardware dominates TCO, make it cheap Must utilize it as well as possible

Virtualization and Operating-System Components

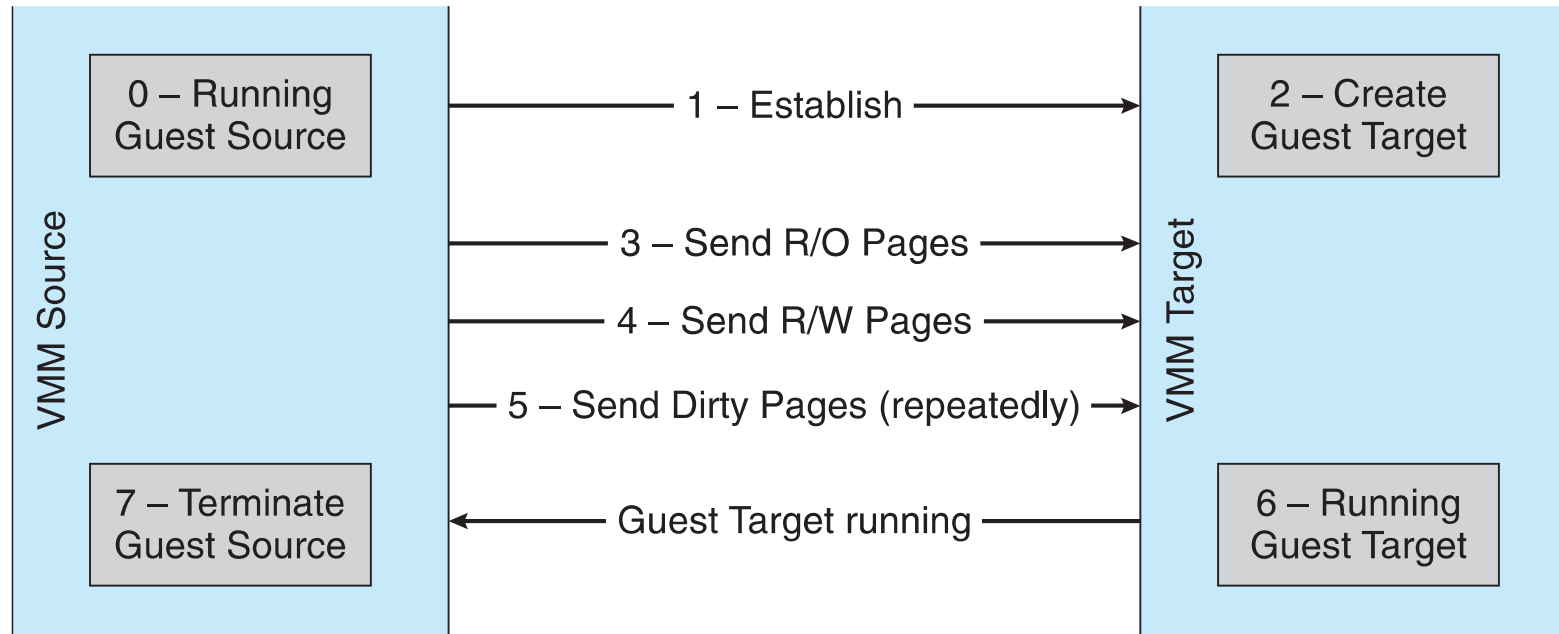
Now look at operating system aspects of virtualization

- CPU scheduling, memory management, I/O, storage, and unique VM migration feature
- Homework:
 - ▶ How do VMMs schedule CPU use when guests believe they have dedicated CPUs?
 - ▶ How can memory management work when many guests require large amounts of memory?

OS Component – Live Migration

- Taking advantage of VMM features leads to new functionality not found on general operating systems such as live migration
- Running guest can be moved between systems, without interrupting user access to the guest or its apps
- Very useful for resource management, maintenance downtime windows, etc.
 1. The source VMM establishes a connection with the target VMM
 2. The target creates a new guest by creating a new VCPU, etc.
 3. The source sends all read-only guest memory pages to the target
 4. The source sends all read-write pages to the target, marking them as clean
 5. The source repeats step 4, as during that step some pages were probably modified by the guest and are now dirty
 6. When cycle of steps 4 and 5 becomes very short, source VMM freezes guest, sends VCPU's final state, sends other state details, sends final dirty pages, and tells target to start running the guest
 - Once target acknowledges that guest running, source terminates guest

Live Migration of Guest Between Servers



In live VM migration, the **pre-copy method** transfers most of the VM's memory while it's still running, sending **read-only pages first** to minimize the final data transfer. This reduces **downtime** and ensures a **faster, smoother migration** with minimal impact on running applications.

Example: VMware vMotion in Action

Scenario:

At El Sewedy University's data center, a server named **ESX-Host-A** runs several virtual machines.

- One VM — “**StudentPortal-VM**” — hosts the student registration system.
- During class hours, the IT team needs to **upgrade hardware firmware** on ESX-Host-A, **without stopping the Student Portal**.



What Happens Step by Step

1. Pre-copy phase:

VMware starts copying the VM's memory pages from Host-A to Host-B **while the VM is still running**.

- Read-only pages are copied first.
- Any pages changed during the process are marked as **dirty**.

2. Iterative copying:

VMware keeps re-copying only the dirty pages that changed since the last round.

- Each iteration transfers less data.

3. Stop-and-copy (final phase):

When very few pages remain dirty, VMware briefly **pauses the VM** on Host-A (milliseconds).

- Copies the remaining pages + CPU state.
- Then **resumes the VM instantly** on Host-B.

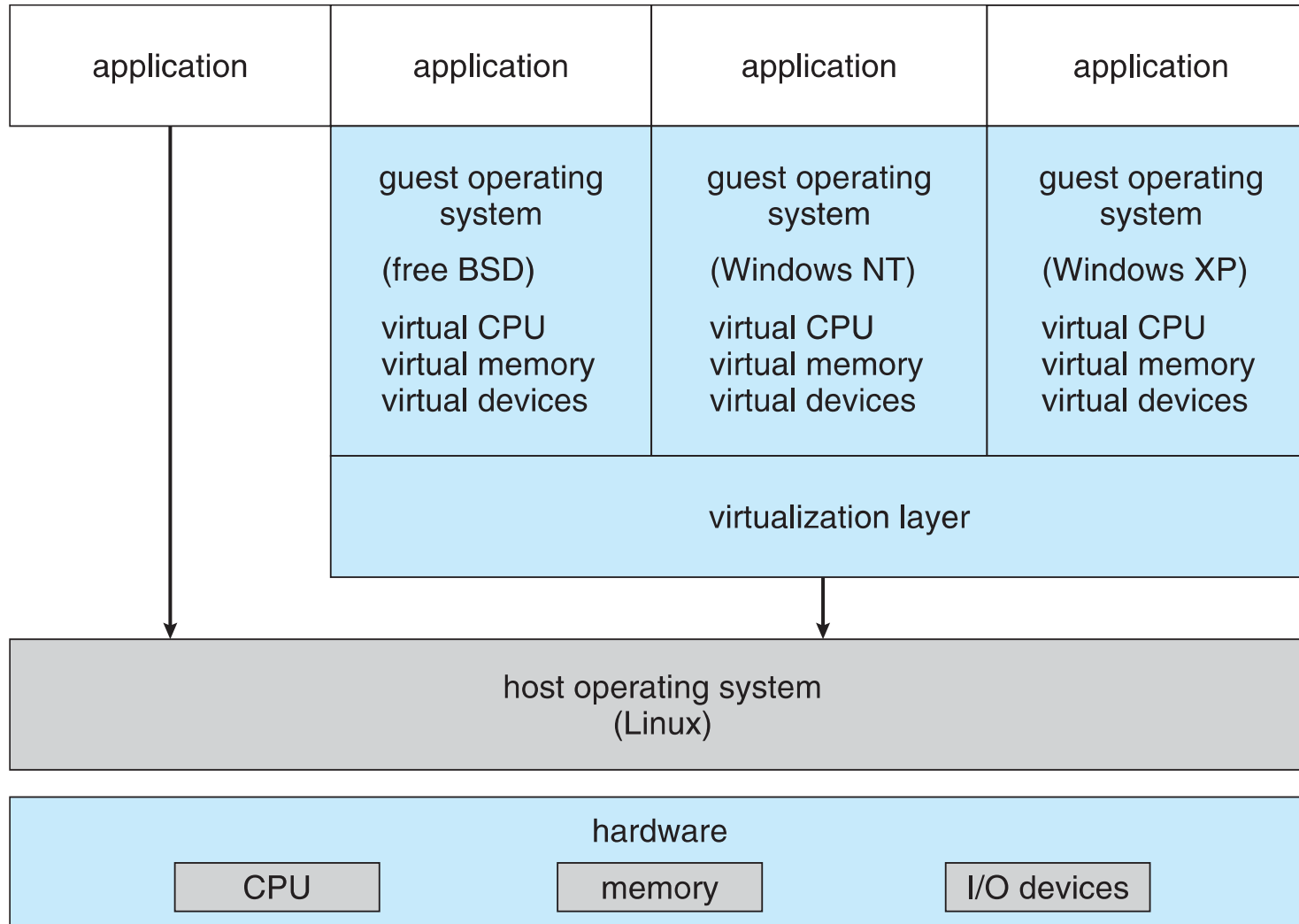
4. Post-copy cleanup:

Host-A releases resources, and users continue using the Student Portal seamlessly.

Examples – VMware (Self Reading)

- **VMware** Workstation runs on x86, provides VMM for guests
- Runs as application on other native, installed host operating system -> Type 2
- Lots of guests possible, including Windows, Linux, etc. all runnable concurrently (as resources allow)
- Virtualization layer abstracts underlying HW, providing guest with its own virtual CPUs, memory, disk drives, network interfaces, etc.
- Physical disks can be provided to guests, or virtual physical disks (just files within host file system)

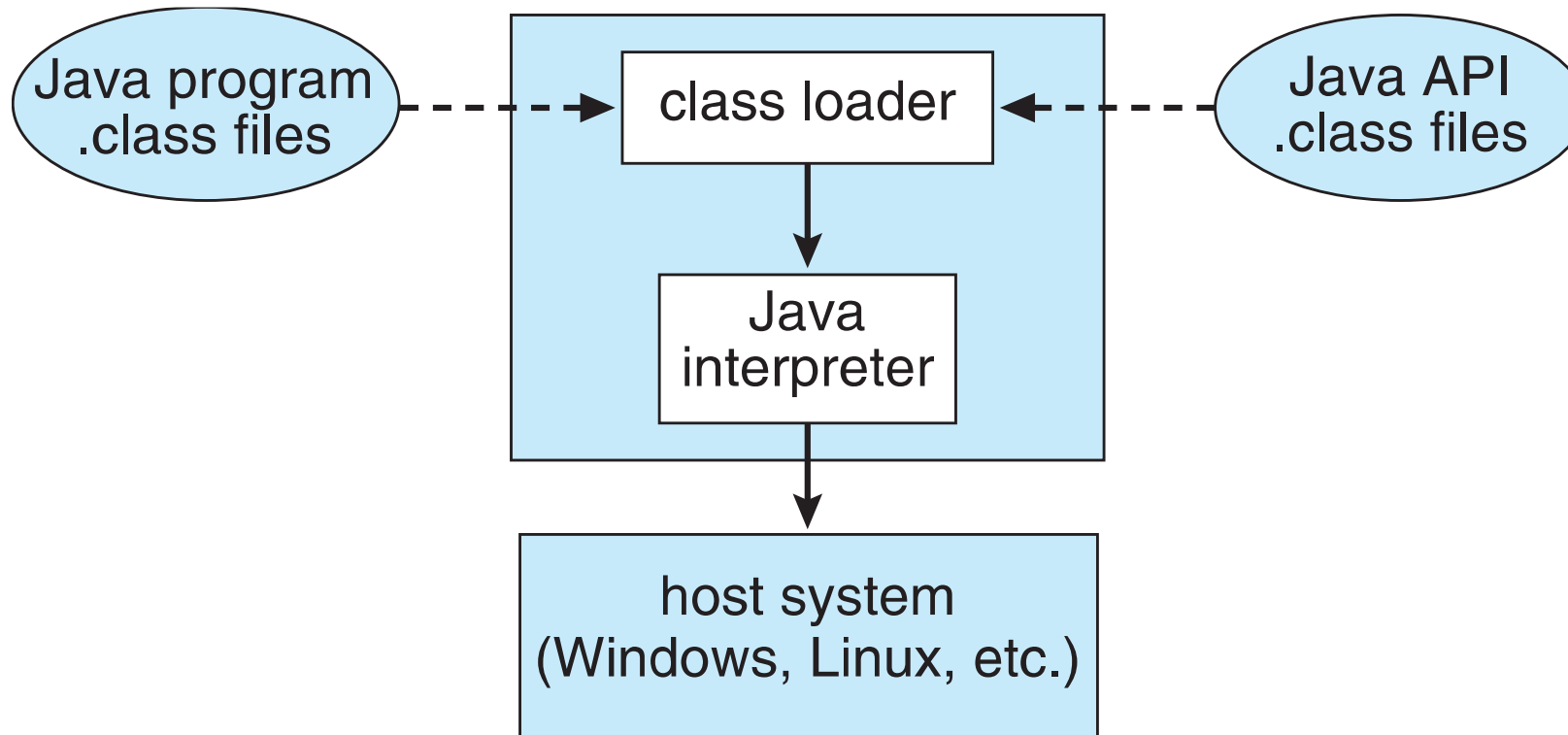
VMware Workstation Architecture



Examples – Java Virtual Machine

- Example of programming-environment virtualization
- Very popular language / application environment invented by Sun Microsystems in 1995
- Write once, run anywhere
- Includes language specification (Java), API library, Java virtual machine (JVM)
- Java objects specified by class construct, Java program is one or more objects
- Each Java object compiled into architecture-neutral **bytecode** output (**.class**) which JVM **class loader** loads
- JVM compiled per architecture, reads bytecode and executes
- Includes **garbage collection** to reclaim memory no longer in use
- Made faster by **just-in-time (JIT)** compiler that turns bytecodes into native code and caches them

The Java Virtual Machine





Suggested student task

1. Install VirtualBox or VMware.
2. Create a VM: 2 CPU, 2 GB RAM, 20 GB disk.
3. Mount Ubuntu ISO and install OS.
4. Run basic commands: ls, pwd, top.
5. Optional: install gcc/python and run a simple program.

Hands-On Project

Build & Explore (TWO) Virtual Machines with a Hypervisor

Review Questions

Lecture 2: Virtualization

- Core Concept: In your own words, what does it mean to “abstract hardware into several different execution environments”?
- Hypervisor Types: What is the difference between a Type 1 and a Type 2 hypervisor? Can you give one example of each?
- Practical Link: Why do cloud providers prefer Type 1 hypervisors?
- Process Reflection: In a Type 2 hypervisor, each VM runs as a process. What happens if the host OS crashes?
- CPU Virtualization: How does the hypervisor make each VM “think” it has its own CPU?
- Trap & Emulate: What is a “trap” in virtualization, and why is it important?
- Protection Rings: Why does only Ring 0 have the ability to run privileged instructions?
- Real-World Use: Why would a company use virtualization for software testing instead of buying multiple physical servers?
- Hands-on Reflection: If you had to create a VM on your laptop, what resources (CPU, RAM, storage) would you allocate to make it usable for programming?
- Advanced Thought: Live migration allows moving a running VM between servers without downtime. Why is this feature critical in cloud data centers?

Examples:

1. Syscall
2. Process creatio
3. Thread creation

End of Lecture 2